CENTRO UNIVERSITÁRIO SENAC PORTO ALEGRE – UNISENAC
Escola de Tecnologia da Informação (Escola de TI)
Componente Curricular: Programação Orientada a Objetos
Docente: Prof. Me. Fábio Giulian Marques

📋 Enunciado: Exercício de Fixação Herança e Composição

🎯 Objetivo

Praticar os conceitos fundamentais de **Orientação a Objetos em Python**, aplicando:

1. **Herança Simples**: Criação de uma superclasse (Animal) e subclasses especializadas (Cachorro, Gato e Passaro).
2. **O Uso do pass**: Compreensão de métodos estruturais vazios (*stubs*) na superclasse.
3. **Métodos Independentes**: Implementação de funcionalidades exclusivas em uma subclasse específica.
4. **Composição**: Relacionamento estrutural em que os animais possuem uma lista de objetos de uma classe independente (Consulta).

🏗️ Descrição do Cenário

Você foi contratado por uma rede de clínicas veterinárias para desenvolver um módulo de cadastro e atendimento de pets. O sistema precisa gerenciar dados genéricos dos animais, permitir a emissão de sons customizados, registrar serviços exclusivos e manter um histórico de atendimentos médicos estruturado.

Para atender aos requisitos, você deverá implementar o seguinte diagrama de classes em Python:

1. A Classe Consulta (Composição)

- **Atributos**: data (str) e hora (str).
- **Métodos**: Um método exibir_consulta(self) que retorna os dados formatados da consulta.

2. A Superclasse Animal

- **Atributos**: nome (str) e idade (int).
- **Atributos de Composição**: Uma lista interna historico_consultas = [] criada diretamente no construtor para armazenar instâncias da classe Consulta.
- **Métodos**:
  - emitir_som(self): Contém apenas a instrução **pass** como um espaço reservado (*stub*), servindo de contrato para as subclasses.
  - agendar_consulta(self, data, hora): Instancia um objeto Consulta e o adiciona à lista de histórico do animal.

- listar_consultas(self): Exibe todas as consultas agendadas para o pet.
- listar_animal(self): Exibe de forma formatada o nome e a idade do animal.

3. As Subclasses Finais (Cachorro, Gato e Passaro)

- **Cachorro**: Herda de Animal, adiciona o atributo exclusivo raca e **sobrepõe** o método emitir_som() para imprimir um latido.
- **Gato**: Herda de Animal, adiciona o atributo exclusivo cor, **sobrepõe** o método emitir_som() para imprimir um miado e **sobrepõe** o método listar_animal() para incluir a cor do pelo.
- **Passaro**: Herda de Animal, adiciona o atributo exclusivo especie e implementa um **método independente** chamado cantar(self) (exclusivo para aves, que não existe nas outras classes).

💻 Esqueleto de Apoio para Codificação

Utilize a estrutura base abaixo para implementar o seu código:

```
class Consulta:
    def __init__(self, data: str, hora: str):
        self.data = data
        self.hora = hora

    def exibir_consulta(self):
        return f"Data: {self.data} | Hora: {self.hora}"


class Animal:
    def __init__(self, nome: str, idade: int):
        self.nome = nome
        self.idade = idade
        self.historico_consultas = []  # Composição

    def emitir_som(self):
        pass  # Espaço reservado (stub) para as subclasses implementarem

    def agendar_consulta(self, data: str, hora: str):
        # TODO: Instanciar Consulta e adicionar à lista historico_consultas
        pass

    def listar_consultas(self):
        # TODO: Percorrer e exibir as consultas agendadas
        pass

    def listar_animal(self):
```

```
        print("-" * 35)
        print(f"Nome do Pet: {self.nome} | Idade: {self.idade} anos")
        print("-" * 35)


# TODO: Implementar a classe Cachorro(Animal)
# - Construtor com super().__init__ e atributo raca
# - Sobrescrita do método emitir_som()


# TODO: Implementar a classe Gato(Animal)
# - Construtor com super().__init__ e atributo cor
# - Sobrescrita de emitir_som() e listar_animal()


# TODO: Implementar a classe Passaro(Animal)
# - Construtor com super().__init__ e atributo especie
# - Método independente cantar()
# - Método listar_passaro() reutilizando listar_animal()

if __name__ == "__main__":
    # TODO: Instancie um cachorro, um gato e um pássaro
    # TODO: Teste a emissão de sons (polimorfismo) e o método independente cantar()
    # TODO: Agende consultas utilizando a composição e exiba os históricos
```

📝 Questões Discursivas de Fixação

1. **Qual é a utilidade prática da instrução pass dentro do método emitir_som na superclasse Animal?**
2. **O que aconteceria se você tentasse chamar o método cantar() a partir de uma instância da classe Cachorro? Justifique com base no conceito de métodos independentes.**
3. **Explique a diferença conceitual entre a *Herança* aplicada entre Animal e Cachorro e a *Composição* aplicada entre Animal e Consulta.**

"Conteúdo gerado com suporte de Inteligência Artificial para estruturação pedagógica e adaptado, revisado e validado tecnicamente pelo Professor."